ProveIt research report

Checked transport and seed-rank proofs for K215

A concrete Context215 equivalence, a stronger enumerator interface, and a scoped search below 215 states

Abstract

This report advances proof simplification at a pinned current ProveIt snapshot. It supplies a concrete Rocq/Coq transport proof for the five-row Context215 patch, based on a seven-state tape invariant and an exact simulation of entire tapes. The invariant's nontrivial preservation obligations are restricted to eleven incoming rows. A separate checked refactor replaces three scheduler inductions by one natural-valued rank and proves completeness of the numerical enumerator from every initial state; soundness retains its necessary derivability invariant. Local opacity removes unnecessary normalization of arithmetic formula codes. The report compares alternative 215-state candidates without conflating paper proofs, local kernel checks, and complete ZFC endpoints. An expanded finite search admits both-branch state folds, checks 22 literal tables, and exhibits a precise valid-frame counterexample to a tempting 214-state construction. No verified machine with 214 or fewer states is claimed. Source files, patches, literal tables, independent audits, build receipts, and reproducible search artifacts accompany the report.

Baseline	`aa66d0a33f1f8237a348899eb8095b045a502832`
Date	21 October 2026
Preparation	Research contribution prepared with AI assistance
Scope	Additive artifacts for review and integration into ProveIt

Contents

1 Results and their exact status **2**

2 A direct transport proof for Context215 **3**

2.1 Tape representation and the invariant 3

2.2 A transient relation on complete configurations 4

2.3 Literal tables, terminal conventions, and attachment 5

3 One rank for the numerical scheduler **6**

3.1 Completeness from every initial enumerator state 7

3.2 Compatibility and measured proof cost 7

3.3 A concrete Lean adaptation 8

4 Prevent unnecessary unfolding of arithmetic codes **9**

5 Which 215-state candidate is easier to prove? **9**

5.1 A compositional family and an exact clock observation 10

6 Search for 214 states: results and an explicit obstruction **11**

6.1 Two-branch folding with unchanged helper interiors 11

6.2 Completed searches and their exact scope 12

6.3 Nineteen partial targets and the 52/154 example 13

6.4 Further research questions 14

7 Integration strategy and further research questions **14**

7.1 Immediate integration order 14

7.2 Questions with concrete next experiments 15

8 Reproduction, provenance, and limits **16**

8.1 What was executed 16

8.2 Artifact layout and replay 16

8.3 Exact machine and source binding 17

1 Results and their exact status

The most effective immediate strategy is to retain the accepted K215 endpoint and simplify reusable proof interfaces around it. The current repository already records compiled external Lean and Rocq equivalences for the literal 215-state machine. Its internal PA endpoint is separately marked open. This report does not change that distinction or assert unconditional nonhalting. The mathematical target remains

$$\mathrm{ZFC} \vdash \big(\neg\,\mathrm{Halt}(M, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{ZFC})\big).$$

A proof assistant's external theorem about a consistency predicate and a formal derivability statement *inside* a particular arithmetic theory are different deliverables. The existing 240-state internal PA theorem is also a separate result [1, 2].

The baseline already incorporates ordinary zero-offset certificates, shared implication facts, AX-word contracts, and semantic-lemma reuse. Three additional ZIP submissions in `docs/incoming` were read before choosing this contribution. In particular, the newest affine-core packet already contains the ordinary six-block Context215 proof and an independent finite checker, while explicitly leaving its exact compiled attachment open [4]. Repeating those calculations alone would not provide a new proof result.

During this investigation, main advanced to `b0663213602b3635258150c58ffe105a2e0ba44f`. Its automatic-certificate packet already supplies further ordinary-certificate compression, automatic source-certificate extraction, concrete zero-credit codec work, and a 21-table predecessor-specific elimination search. Its latest Lean change reuses the semantic round-trip result. These are credited as existing work, not proposed here as new contributions [6]. The three edited Rocq modules and the two literal sparse-machine modules were individually fetched at this newer pin and have exactly the same Git blobs as at the baseline. The refresh was targeted; no complete audit of all intervening files is claimed.

Contribution	What is established	Boundary
Concrete transport	Exact old/new native tables, arbitrary initial tape at control 0, complete tape preservation, positive simulation	Kernel checked through the actual sparse K215 halting predicate
Ranked scheduler	Bounded search for any chosen candidate; completeness from every initial enumerator state	Checked against the exact numerical calculus and compiler
Soundness opacity	Existing soundness statements and scripts survive local opacity of arithmetic constructors	A measured module improvement, not a whole-project benchmark
Alternative comparison	Distinguishes shortest mathematical argument from least remaining integration work	Grounded in inspected current sources and incoming packets
State-fold search	A complementary both-branch fold template and a reproducible failed 214-state trial	A bounded search, not a minimality theorem

The numerical state count is unchanged by the proof refactors. Likewise, compressing a source graph or a proof certificate does not by itself shorten the physical Turing machine. The archive maintains those quantities separately.

2 A direct transport proof for Context215

Let A be the accepted native K215 table and B the Context215 table. Native start control is 0; the separate terminal control is 254. Each table has 215 working controls and 430 binary transition rows. The five changes are all retargetings: their written bit and head direction agree with the corresponding old row.

Control	Read	Old action	New action
13	1	$1\,L\,31$	$1\,L\,81$
27	1	$0\,L\,40$	$0\,L\,302$
83	1	$0\,L\,42$	$0\,L\,57$
156	1	$1\,R\,176$	$1\,R\,52$
198	0	$1\,L\,40$	$1\,L\,54$

These table changes are inherited from the existing Context215 contribution [3]. The new formalization uses an inductive tape invariant and a short transient-state relation. It does not rerun the 6,176 primitive contracts, the source-program correspondence, or the numerical-calculus calibration.

2.1 Tape representation and the invariant

A configuration is (q, L, b, R), where q is the control, b is the scanned bit, and L, R are infinite binary streams, nearest cell first. A transition removes the nearest bit from the direction of travel and prepends its written bit to the opposite stream. Thus a local calculation can prove equality of the entire resulting configuration by constructor equality. No finite truncation of the exterior tape is assumed. The native development uses no stream extensionality axiom.

Write ℓ_0, ℓ_1 for the first two bits of L, and r_0 for the first bit of R. Define J by the following clauses; at every unlisted control it is true.

Control	Required condition
13	$\ell_0 = 1 \ \vee \ [\ell_0 = 0 \wedge (\ell_1 = 1 \vee b = 0)]$
27	$\ell_0 = 0$
38	$\ell_0 = 1$
42	$\ell_0 = 1 \vee r_0 = 0$
83	$\ell_0 = 1$
156	$r_0 = 1$
198	$\ell_0 = 1$

Lemma 2.1 (Inductive guard invariant)**.** *Every old-machine configuration reached from control 0, with an arbitrary initial tape and scanned bit, satisfies J.*

Proof. The initial assertion is immediate because control 0 is unlisted. Every transition whose destination is unlisted trivially preserves J. Filtering the complete old table by the seven listed destination controls leaves exactly eleven rows:

$$8/1 \to 13, \quad 26/1 \to 38, \quad 28/1 \to 42, \quad 38/0 \to 13, \quad 42/0 \to 13,$$
$$66/0 \to 198, \quad 73/1 \to 83, \quad 83/1 \to 42, \quad 95/0 \to 27, \quad 109/1 \to 42, \quad 189/1 \to 156.$$

The kernel checks that this list is precisely the filtered literal table. Each remaining obligation is a finite stream calculation.

For example, an entry to 13 from 38 writes zero and moves right. The previous control-38 clause supplies the one at the next cell to the left. An entry from 42 also writes zero and moves right. Its old clause supplies either the required left one or a zero at the newly scanned cell. The latter possibility cannot trigger the patched read-one row. The clause at 42 itself follows either from an incoming right-moving write-one, or from 83/1, which writes zero and moves left. Induction over old steps proves the claim. □

This proof needs only the old-machine invariant. A separate new-machine history classification is unnecessary. The finite filtering step also avoids a large tactic proof with 430 repeated row obligations: the generic noncritical-row lemma handles all other rows at once.

2.2 A transient relation on complete configurations

Relate identical configurations. In addition, relate the following old/new control pairs with *identical full tape streams and head position*:

Old	New	Tape guard	New steps per old step
31	81	$b = 0,\ \ell_0 = 1$	3
31	81	$b = 1,\ r_0 = 1$	1
47	57	$b = 1$	1
40	302	$b = 0$	1
42	57	$b = 1$	1
176	52	$b = 1$	1
40	54	$b = 1$	1

The first case is the only delayed excursion. The old read-zero transition at 31 reaches 21 in one step. The new execution traverses

$$81 \longrightarrow 111 \longrightarrow 128 \longrightarrow 21$$

in three steps. The extra left excursion reads the known one, preserves it, and returns across the known zero. Its endpoint is exactly the old endpoint. The read-one 31/81 case proceeds to the 47/57 case, which then synchronizes. Every other pair synchronizes in one step.

Lemma 2.2 (Positive-step simulation)**.** *If $J(c)$ and $c\ \mathcal{R}\ d$, then for some $k \in \{1, 2, 3\}$,*

$$\mathrm{step}_A(c)\ \mathcal{R}\ \mathrm{run}_B(k, d).$$

In the concrete proof k is always 1 or 3. Moreover, c has control 254 if and only if d does.

Proof. For an identical pair, an unchanged row produces identical successors. Each changed row enters the corresponding transient pair; J supplies its tape guard. For a transient pair, the literal transitions give the complete configurations described above. None of the nonidentity pairs uses control 254, so halting reflection follows immediately. □

Theorem 2.3 (All-tape Context215 equivalence)**.** *For every pair of initial streams L, R and scanned bit b,*

$$\mathrm{Halt}(A, (0, L, b, R)) \quad \Longleftrightarrow \quad \mathrm{Halt}(B, (0, L, b, R)).$$

Consequently the blank-tape nonhalting predicates are equivalent.

Proof. Iteration of the preceding lemma gives, for every n, a natural m with

$$n \le m \le 3n, \qquad \mathrm{run}_A(n, c_0)\ \mathcal{R}\ \mathrm{run}_B(m, c_0).$$

Old halting at n therefore gives new halting at m.

For the reverse direction, suppose the new machine halts at time n. Apply the simulation to *old* time n. It gives a related new configuration at some time $m \ge n$. Terminal control 254 is absorbing, so that new configuration still has control 254. Reflection through $\mathcal{R}$ implies that the old configuration at time n has control 254. This proves both directions without constructing a separate reverse simulation. □

The positive lower bound on the simulated duration is essential to this argument. A relation permitting arbitrarily many old steps to correspond to zero new steps would not supply $m \ge n$ and would not justify this reverse implication.

2.3 Literal tables, terminal conventions, and attachment

`NativeContext.v` contains the old table as a literal list. `LiteralBindings.v` contains all 430 rows of the new table and proves that they equal the rowwise patch of the old table. It also checks the working-state count, row counts, unchanged working-label list, and closure of transition targets. The independent byte audit binds these literals to the submitted native files and their SHA-256 values.

The native interpreter totalizes an absent row by leaving its configuration unchanged, and defines halting as reaching 254. Agreement with the intended binary-machine semantics requires the domain obligation: every reachable nonterminal control must have both rows. The complete tables have that property, and all their targets are working labels or 254. The adapter to the existing BusyCoq semantics must retain this obligation because BusyCoq interprets an absent row as halting. An unproved correspondence between these conventions would be a real gap.

The exact semantic attachment is now supplied in `adapter/NativeSparseAdapter.v`. The map from native labels to the maintained sparse labels is

$$301 \mapsto 102, \qquad 302 \mapsto 120, \qquad 254 \mapsto 230,$$

with all other labels unchanged. A finite Boolean certificate checks transition correspondence, active-label bounds and the unique active terminal. A separate certificate checks closure of targets. The active invariant excludes the fifteen disabled sparse working labels. The two stream types are related by a coinductive observational relation; no stream or function extensionality axiom is needed.

The compiled theorem in `adapter/ContextLiteralSparse.v` is exactly

```
context_literal_blank_halts_iff_sparse :
  NativeContext.halts LiteralBindings.literal_new NativeContext.blank <->
  K215F.TM230.TM230.halts K215F.TM230.tm K215F.TM230.TM230.c0.
```

This is a concrete theorem about the supplied literal table and the actual maintained endpoint predicate, with no generic simulation or calibration premise. All ten minimal dependency modules were compiled, and `coqchk` recursively accepted `ContextLiteralSparse`. The selected native, literal-binding, adapter and composition theorems report closed assumptions.

The final ZFC composition is provided separately in `adapter/ContextZFCClient.v`. It imports the actual maintained `K215F.Correct215` endpoint

```
tm215_sparse_halts_iff_ZFC_inconsistent
```

and would give

$$\mathrm{Halt}(B, \mathrm{blank}) \iff (\mathrm{StandardZFC} \vdash_{TC} \bot),$$

then the corresponding negated equivalence. It postulates no foundational theorem. However, this client and the complete 237-source ZFC closure were *not* freshly compiled here. Its composition is explicit and mathematically immediate from the checked adapter and the existing endpoint, but it must not be advertised as a newly executed end-to-end build. Compile that small client in the maintained full K215 lane to record the final integration check.

3 One rank for the numerical scheduler

The second contribution changes the enumerator proof rather than a transition row. Let $\mathrm{np}(s)$ be the next-candidate counter and $\mathrm{pl}(s)$ the current proof-list pointer. The existing exact `step_pl` lemma implies that every successful step has one of two forms:

1. If $\mathrm{pl}(s) > 0$, then np is unchanged and $\mathrm{pl}(s') \le \mathrm{pl}(s) - 1$.
2. If $\mathrm{pl}(s) = 0$, then $\mathrm{np}(s') = \mathrm{np}(s) + 1$ and $\mathrm{pl}(s') \le \mathrm{np}(s)$.

These are arithmetic facts about the existing candidate decoder, not assumptions about whether the current formula or stack entries are derivable.

Fix a candidate number N and assume $\mathrm{np}(s) \le N$. Define

$$\boxed{R_N(s) = (N - \mathrm{np}(s))(N + 1) + \mathrm{pl}(s).}$$

The intended boundary is $\mathrm{pl}(s) = 0$ and $\mathrm{np}(s) = N$.

Lemma 3.1 (Bounded candidate search)**.** *From a state satisfying* $\mathrm{np}(s) \le N$*, at most* $R_N(s)$ *successful steps reach either a state whose next step halts or the boundary for candidate* N*.*

Proof. A draining step leaves the first summand unchanged and decreases the second by at least one. A nonboundary refill has $\mathrm{np}(s) < N$, so $\mathrm{np}(s') \le N$ and

$$R_N(s') - R_N(s) \le -(N + 1) + \mathrm{np}(s) < 0.$$

Thus every successful nonboundary step strictly decreases a natural-valued rank. Strong induction on that rank gives the stated bound, allowing an earlier halting step as an alternative. If one counts the final halting transition itself, the bound is $R_N(s) + 1$; the theorem's displayed bound counts the successful prefix to the state that takes that transition. □

The theorem `find_candidate_bounded` replaces separate drainage, zero-seed reachability, and seed-4 reachability arguments. This is also a better interface for future startup optimization: the scheduler proof no longer requires the actual bootstrap to leave an empty proof-list pointer or one specific next-candidate number.

3.1 Completeness from every initial enumerator state

The existing numerical compiler gives, from a derivation of falsity code 1, a finite command word L satisfying its block contract. Its existing `program_halts` theorem is already independent of the initial working registers. A self-initializing wrapper has the form

$$\mathrm{START}(Y) \ :: \ L +\!\!+ \mathrm{SUFFIX}.$$

The exact candidate encoding used by the proof is

$$N = P\big(3, P(Y, P(1, \mathrm{enc_tail}(L +\!\!+ \mathrm{SUFFIX})))\big).$$

Taking $Y = \mathrm{np}(s)$, the inherited Cantor-pairing inequalities give $N \geq Y$. The rank lemma therefore reaches this halting word or an earlier halt from every initial state.

Theorem 3.2 (Arbitrary-state completeness, checked)**.** *For the exact current numerical calculus and enumerator,*

$$\forall s, \qquad \mathrm{thm222}(1) \implies \mathrm{halts_from}(s).$$

No restriction on the initial proof-list pointer, candidate counter, arithmetic counter, formula, stack, or cache is required for this implication. This assertion is stronger than the previous seed-specific completeness theorem.

The converse is different. The existing soundness invariant $\mathrm{Inv}(s)$ requires derivability of the current formula, the cache, and every entry in the stack stream. Combining it with the new theorem gives the checked interface

$$\forall s, \quad \mathrm{Inv}(s) \implies \big(\mathrm{halts_from}(s) \leftrightarrow \mathrm{thm222}(1)\big).$$

It imposes no extra condition on pl, np, or the arithmetic counter. In particular, the proof covers all states `mkState p n 0 0 d 0` for arbitrary naturals p, n, d.

The invariant must not be omitted. For example, an initial cache containing falsity code 1 can cause a NIP halt without establishing that the code was derivable. The unconditional new theorem is completeness; the equivalence is conditional on the ordinary soundness invariant.

3.2 Compatibility and measured proof cost

The typed client checks the exact existing types of `enum_complete`, `seedPerm_complete`, `seedPerm_sound`, and `seedPerm_iff_thm`. The production `MainPerm.v` composition consumes the unchanged final interface. The audited generated production tree has no other consumers of the removed private scheduler helpers.

Source scope	Before	After
Compiler and seed, bytes	18,100	17,209
All three edited files, bytes	22,105	21,335
Seed file, lines	59	24

The compiler file grows because it proves a stronger result; the combined source shrinks. The more significant improvement is eliminating separate scheduler induction patterns and exposing a reusable theorem for arbitrary initial states. Lines and bytes alone do not measure conceptual proof cost.

All ten required project modules and one typed client were freshly compiled under Coq 8.18.0, with six closed assumption reports and a successful recursive `coqchk`. The only compatibility transformation in private build copies changes the standard-library import spelling `Stdlib` to `Coq`. No compatibility axiom is introduced. The complete binary-machine-to-ZFC closure was not rebuilt for this refactor.

For integration, update both the maintained `rocqperm/` templates and the generated `rocq/rocq-extension/coq/215/` files. Template imports use `NS` where the generated proof copies use `K215F`. Regenerate metadata and run the normal production endpoint build after applying the edits.

3.3 A concrete Lean adaptation

The corresponding Lean sources are under `Computability/BusyBeaver/Lean/BusyBeaver/ZFC/`. Their current `EnumPerm.lean` and `SeedPerm.lean` were inspected at the refreshed pin; exact Git blobs are recorded in `lean-sources.json`. No new Lean implementation or compilation is supplied here [7].

In namespace `SetTheory.BusyBeaver.ZFC.EnumPerm`, the state `St` has fields `pl`, `np`, `tw`, `ws`, `d`, and `h`. Its `HaltsFrom` convention is the same successful-prefix convention used above. The source already exposes the relevant interfaces:

- `step_eq`, `decode_le`, and the definition of `fetch` supply the pointer-decrease and refill bounds.
- `run_succ` and `run_add` compose finite executions.
- `build` supplies a command-list contract from a derivation; `program_none` proves the self-initializing word halts from every working-register tuple.
- `encL_ge` supplies the cofinal code bound, and `exec_list` transfers the compiled command word to `HaltsFrom`.

The existing `drain` and `reach` prove two scheduler inductions. The existing `complete` and `halts_iff` require `s.pl = 0`. Porting `find_candidate_bounded` removes that premise by one strong induction on $R_N(s)$. The proposed stronger interfaces, using the current falsity code, are

```
complete_from (s : St) : C222 Code254.F -> HaltsFrom s
Inv (toW s) -> (HaltsFrom s <-> C222 Code254.F)
```

The current public statements can remain as wrappers.

A useful Lean proof detail is to normalize the refill arithmetic explicitly. With $p = \mathrm{np}(s) < N$, rewrite

$$N - p = (N - (p + 1)) + 1$$

and distribute multiplication. Both old and new ranks then share the summand $(N-(p+1))(N+1)$, so cancellation leaves the linear bound $\mathrm{pl}(s') < N + 1$. Do not ask linear arithmetic automation to invent a nonlinear multiplication identity. The draining case only cancels an unchanged first summand.

For completeness, choose $Y = \mathrm{np}(s)$ and compile the existing word

```
.ax 0 1 Y :: (p ++ [.drop, .nip, .nip])
```

using `build`, `encL_ge`, the new bounded lemma, `program_none`, `exec_list`, and `run_add`. Soundness continues to use `Inv (toW s)`. This is a direct port of the checked Rocq argument, with the actual Lean interfaces identified.

Lean's seed layer is already small: `EnumPerm.seed` is $\langle 0, 4, 0, 0, 0, 0 \rangle$; `seed_iff` is its consistency-calculus interface; `L2Perm.dagPerm_seed_iff` composes `enc_seed`, `iter_iff`, and that interface. It does not duplicate the scheduler induction in the way the old Rocq seed source did. Preserve these statement types rather than claiming a second Lean seed-proof reduction that is not present.

The refreshed upstream `KSeedModel222.lean` already replaces an encoding-round-trip proof route with `Code254Models.sat_encode_decode`, uses `derivable_named_bundle`, and preserves `seed_eqFree`. That semantic simplification is credited as completed upstream work. The proposed rank adaptation addresses the remaining scheduler interface independently.

4 Prevent unnecessary unfolding of arithmetic codes

The soundness module uses abstract constructor and inverse lemmas. It does not need to normalize the arithmetic implementation of formula codes. The declaration

```
Local Opaque P wim wal wex weq wel.
```

prevents incidental conversion from expanding those definitions during its proofs. Every existing theorem statement and proof script in that module remains unchanged. The underlying mathematical constants retain their definitions; local opacity introduces no axiom.

Observed `L1_SoundPerm.v` build	Seconds
Initial unmodified module	43.290
Isolated opacity trial	0.810
Final fresh focused closure	0.665

These measurements are local observations on Coq 8.18 on the execution host. They are not a controlled whole-project benchmark and do not establish a particular production Rocq 9.1 speedup. The complete final focused compilation took 16.996 seconds in summed subprocess times; its separate recursive kernel check took 66.483 seconds. Raw logs and toolchain information are included.

The same engineering principle applies to Lean: expose small constructor and eliminator interfaces, and keep large arithmetic definitions out of routine simplification sets. A corresponding Lean change should be measured and compiled locally before claiming a speedup; this report does not label an unexecuted Lean port as checked.

5 Which 215-state candidate is easier to prove?

There are two distinct comparisons. To simplify the *complete established endpoint*, primary K215 is the preferred target because its substantial calibration and compiler proofs already exist. To add another endpoint, one should minimize the additional adapter obligations, which may favor a near-identical table over a conceptually simpler source language.

Candidate	Advantage	Status in the inspected repository and incoming records
Primary K215	Existing compiled Lean and Rocq endpoints; direct target of the rank refactor	Rebuild production closure after applying changes
M124, FM215, M125	Small inspected local invariant, with two marker clauses	Incoming packet already has generic compiled Coq instances; native-to-repository adapter is separate
CM215	Actual Core/dense bridge is recorded compiled	Final ZFC adapter and its complete closure are explicitly not freshly compiled in the reviewed packet
Context215	Five row changes; eleven-row invariant slice; transparent complete-tape simulation	This report adds the checked concrete transport and exact sparse-semantics attachment
ZC215	Clean independent stack-credit scheduler argument	Current incoming refinements already check the real Cantor codec and occurrence injection; full physical/source attachments remain
GC215/GC-ZC215	Counter-free generalization, optionally combined with the credit argument	Source and runtime arguments are substantial, but there is no complete new kernel endpoint in the reviewed material

These alternatives are not interchangeable proof-status labels. A compiled generic transport theorem is not automatically a compiled concrete instance. A concrete instance in a custom tape semantics is not automatically attached to the repository's endpoint. A recorded earlier build is also different from a fresh local build in this report.

Primary K215 starts with a zero stack and does not need the arbitrary-tail debt theorem that motivates ZC215. Consequently it would be misleading to advertise ZC215 as deleting such a debt argument from the existing primary proof. Its benefit is an independent scheduler design and a potentially cleaner route for different bootstrap conditions.

The lower-risk recommendation is therefore to integrate the rank/opacity refactor into primary K215 and attach Context215 through exact table transport. For a second 215-state endpoint, Context215 is now the most concrete recommendation from this investigation because that additional semantic attachment has actually been checked. If the criterion is the shortest human local argument, M124/FM215/M125 have the smaller two-marker invariant. Both routes inherit the difficult foundational calibration from primary K215; neither independently removes it. A more radical calculus or source-language change should be justified by a quantified reduction in proof obligations or by an actual physical state saving.

5.1 A compositional family and an exact clock observation

All five altered-block interiors avoid every altered row. Applying any subset of the retargetings therefore preserves the same ordinary all-tape halting argument: selected blocks use the new path, omitted blocks retain the old transition. An independent finite audit verifies all 32 masks, each with 215 graph-reachable working controls. Their rooted read-labelled write/move graphs are pairwise nonisomorphic, including under global tape reflection.

This is a compositional corollary of already published retargetings, not a claim to 32 historically new machine architectures. The complete five-patch concrete instance is kernel checked here; the all-mask family is reported at the ordinary proof plus independent finite-audit level.

At synchronized block boundaries, the old/new step counts satisfy

$$T_S = T + 2N,$$

where N counts completed read-one state-13 blocks with the zero predecessor, provided that patch is selected. Other blocks have equal durations. No strict block interior halts, so the same identity holds for halting times. Since each delayed old block consumes two steps, $T \le T_S \le 2T$. This sharper bound is a separate ordinary block-count corollary. The kernel theorem presented in Section 2 supplies the coarser $n \le m \le 3n$ bound and should not be cited as proving the sharper one.

6 Search for 214 states: results and an explicit obstruction

No verified machine with 214 or fewer working states was obtained. The contribution is a more general exact local fold template, a reproducible search scope, and a concrete counterexample that explains why one apparently attractive 214-state table does not satisfy the accepted uniform compiler contract.

All input tables are pinned to repository commit `aa66d0a33f1f`. The full commit identifier, repository paths and SHA-256 values are in `state_reduction/inputs/source-index.json`. The corpus consists of 22 byte-distinct 215-working-state tables; byte distinction is not a claim that all 22 rooted transition graphs are pairwise nonisomorphic. These searches do not establish a lower bound on the size of a machine realizing the desired consistency equivalence.

Upstream refresh. Before packaging, we reviewed the predecessor-specific elimination packet integrated at commit `b0663213602b`. Its grammar edits only external rows whose old target is the deleted control, permits different replacements for those rows, and admits later exact old/new joins. The folding grammar here redirects the deleted control to one retained control and may change both retained rows, even when they did not previously target the deleted control, while freezing helper interiors. These are complementary restricted grammars with overlap. The finite-window method and positive-simulation argument are shared ideas, without a historical-priority claim. All 21 upstream table hashes occur in our corpus; FM215 is our additional input. Exact reviewed-file hashes and corpus overlap appear in `state_reduction/evidence/upstream-refresh.json`.

6.1 Two-branch folding with unchanged helper interiors

Let T be a deterministic binary transition table, total on its working controls, with separate halt H. A configuration is (q, h, τ), where $\tau : \mathbb{Z} \to \{0, 1\}$. Choose distinct working controls q, r, with r different from the initial control. Let $\phi(r) = q$ and let ϕ fix every other control, including H. The candidate deletes r, redirects every target r to q, and changes only the two rows of q.

Assume that, for each read bit b, the old rows have common write and movement components:

$$T(q, b) = (w_b, d_b, t_{q,b}), \qquad T(r, b) = (w_b, d_b, t_{r,b}), \qquad d_b \in \{-1, 1\}.$$

Their targets need not agree for either bit. Let W_p be an inductive set of local tape windows at old control p. It must include every window at the initial control and be closed under writing the center, shifting with the head, and admitting both possible fresh exterior bits. Induction on the concrete run proves that every reachable old configuration has its window in W_p. This is an overapproximation; a member of W_p need not be concretely reachable.

Folding proposition. For each bit b, suppose one fixed replacement action a_b has the following property for both $p = q$ and $p = r$, on every admitted window whose center is b. Starting at control q with that full tape, the candidate performs a positive finite block ending at

$$(\phi(t_{p,b}),\ h + d_b,\ \tau[h \mapsto w_b]).$$

The block reads no unspecified exterior cell and every strict interior control avoids q, r, H. Then, from the initial control, the candidate and T halt equivalently on every initial binary tape and at every initial head position.

Proof. At an old control outside $\{q, r\}$, one candidate transition gives the image of the old successor under ϕ, with identical tape and head. At either folded control, the assumed block gives precisely the same property after a positive number of candidate transitions. The exclusion of changed controls from strict interiors ensures that an old helper execution remains an execution of the candidate after target renaming; no unchecked recursive use of a changed row is hidden inside a certificate. The window invariant supplies the hypotheses at every old reachable boundary. Concatenation gives a positive-duration simulation preserving the halt control and containing no halt in a strict interior. If the old run halts, its finite block expansion halts. If it is infinite, the positive block lengths produce arbitrarily long prefixes of the deterministic candidate run without halting. This proves both halting directions. No consistency assumption is used. □

Exact three-transition instance. For current bit b, select an existing helper u_b. Let m be the predecessor symbol at offset $-d_b$. It suffices, for every permitted origin/marker pair, to have

$$T(u_b, m) = (m, d_b, s), \qquad T(s, w_b) = (w_b, d_b, t), \qquad \phi(t) = \phi(t_{p,b}),$$

with $u_b, s \notin \{q, r, H\}$. Set

$$T'(q, b) = (w_b, -d_b, u_b).$$

The head goes backward, forward to the written cell, and forward to its required final position. The marker is preserved and the only net write is w_b at the original head. A direct one-transition branch is also permitted when the two mapped targets agree.

Why the branch obligations factor. The replacement action for read zero never executes inside a read-one certificate, or conversely: strict interiors avoid the retained control entirely. Consequently the two rows can be synthesized independently and combined whenever both admissible-action sets are nonempty. This removes the previous requirement that one branch already be identical. It also avoids a quadratic search over pairs of row assignments. The broader helper-walk implementation computes valid literal entry-action profiles first, then intersects them for each proposed state pair and rejects profiles whose interiors touch either merged control.

The broader implementation uses literal endpoint profiles, an explicit restriction: it does not identify two distinct first-action targets merely because a later quotient would identify them. Its recorded scope must be retained when interpreting a no-result outcome.

6.2 Completed searches and their exact scope

The first independent implementation checks the direct/three-transition instance with a radius-three invariant. Across 22 input tables it examines 468,218 ordered retained/deleted pair choices satisfying the common write/move condition. In 458,714 choices, both *mapped* target branches differ. No complete fold is found.

The second implementation permits frozen-helper walks of up to 64 transitions using a radius-five invariant, hence an eleven-cell known window. It considers the same 468,218 ordered pair choices. A profile must preserve the exact old write, head displacement and literal target for every admitted old window. The two selected profiles must avoid the merged controls in their strict interiors. There are 26,458 successful individual profiles across the corpus, but no state pair has compatible complete profiles for both branches. Reading outside the known window is recorded as unresolved; it is not a claim that a longer or wider proof is impossible. These two search totals overlap and must not be added as independent candidate counts.

A nontrivial positive control tests the mechanism. A synthetic 216-state table splits the old origin markers between controls 69 and 400, and gives them different targets for *both* input bits. The search recovers

$$69/0 = 0L213, \qquad 69/1 = 1L213$$

after folding 400 into 69. Four independent literal three-transition replays verify both marker values and both current bits, with arbitrary untouched exterior tape. This fixture checks the implementation; it has no asserted ZFC calibration.

A separate experiment combines the existing history-return table with the inherited contextual-helper resynthesis algorithm. It examines 3,143 helper specifications, with 1,435 decoder-prefix passes and 480 delivery-class passes. All 480 fail the symbolic runtime prover. Their failure reasons include an undischarged possibly-empty run, entry into opaque context, or a derivation that does not close. These are *proof failures*, not 480 independently established semantic counterexamples. The newly explicit counterexample below has a stronger status.

6.3 Nineteen partial targets and the 52/154 example

The new two-branch search records nineteen unordered primary-K215 pairs for which both mapped targets differ but one branch already has a three-transition helper. Their exact rows and helpers are retained in `partial-two-branch-targets.json`. Thus the expanded search supplies concrete missing obligations rather than only a negative count.

The pair $(52, 154)$ is particularly transparent. The old rows are

	0	1
52	$0R154$	$1R150$
154	$0R166$	$1R12$.

Every old incoming edge to 52 writes one and moves right; every old incoming edge to 154 writes zero and moves right. Their predecessor markers therefore distinguish the two roles on every run entering from the initial control. For the zero branch, helper 211 already suffices:

marker 0	$211 \xrightarrow{0R} 159$	$159 \xrightarrow{0R} 166$
marker 1	$211 \xrightarrow{1R} 129$	$129 \xrightarrow{0R} 154$.

After redirecting 154 to 52, this gives the required mapped endpoint in both cases.

The one branch must instead end in 12 for marker zero and in 150 for marker one. State 117 has the useful existing zero row $117/0 = 0R134$, followed by $134/1 = 1R12$. Changing its other row to $117/1 = 1R176$ would supply $176/1 = 1R150$ on the marker-one side. This suggests the concrete trial

$$52/0 := 0L211, \qquad 52/1 := 1L117, \qquad 117/1 := 1R176,$$

with every target 154 redirected to 52 and control 154 removed. It has exactly 214 working states and 428 transition rows. All four intended selector blocks are correct, and the independent verifier checks them as whole-tape local calculations.

However, old row $117/1 = 0L302$ is a live CB1 entry. Borrowing it is unsound for the inherited runtime interface. Use the accepted source site of zero-based ordinal 4724,

$$\texttt{11011}, \qquad \mathrm{CB}, 1,$$

with all nine logical registers zero, gap zero and a single zero-valued spare. This is a valid uniform runtime frame. Its initial head is zero and its one cells are

$$\{-1, 0, 1, 3, 4, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84\}.$$

The parent returns to a clean dispatcher frame after 163 transitions, with head -1 and one cells

$$\{-2, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84\}.$$

The trial returns after 588 transitions, with head zero and one cells

$$\{-1, 0, 1, 3, 4, 8, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84\}.$$

The discrepancy is an exact finite counterexample. The independent verifier checks the input hashes, reconstructs the trial from its three-row recipe, checks the source ordinal, and replays all transitions. It imports no symbolic runtime, allocator, or search module. This refutes the trial's claimed uniform primitive simulation; it makes no statement about the trial's blank-run halting behavior or consistency equivalence.

The practical research target is consequently precise: realize the missing one-branch selector while preserving the live CB1 role of 117, perhaps by a coordinated relocation of that role. Merely proving the four new selector blocks does not discharge this obligation. The supplied trial is named `REJECTED-52-154-214.native.tm` and must remain outside the accepted-candidate inventory.

6.4 Further research questions

Can the nineteen partial two-branch folds be completed using coordinated helper relocations whose old and new uses share one verified block certificate? Can a compiler choose source associations specifically to make the required origin markers available, while pricing every relocated primitive entry? Can the branch-profile search use a richer inductive tape abstraction without recreating the full symbolic primitive proof for every rejected table? Finally, can the positive-stuttering proposition be exposed as a small certified optimizer interface, so a successful candidate carries a finite, kernel-checked transport proof from an existing exact endpoint? These questions retain the actual remaining obligations instead of replacing them by a source-DAG count or a successful blank prefix.

These computations do not prove that K215 is minimal. They also do not exclude a fold needing a reachable-tape invariant, a longer return, an altered source encoding, a different initializer, or a different numerical calculus. A failed symbolic proof search is especially weak negative evidence unless accompanied by an explicit contradicting run.

7 Integration strategy and further research questions

7.1 Immediate integration order

1. Apply the three generated Rocq source patches and their corresponding maintained template changes. Preserve the existing public seed interfaces, refresh generated-file provenance, and rebuild the production endpoint.

2. Add the native Context215 transport and its literal table bindings as a small independent component. Keep the raw table hashes and domain checks alongside the proof.

3. Use the supplied checked native-to-sparse semantic adapter, then compile its small ZFC composition client inside the maintained full dependency closure. Record which adapter and endpoint modules were freshly compiled rather than inheriting a broad status label.

4. Port the single-rank enumerator theorem to Lean against the current `EnumPerm`/`SeedPerm` definitions. Prove the exact pointer-decrease and refill inequalities once and reuse the existing compiler's reset wrapper.

5. Keep unsuccessful physical candidates in a clearly labelled research directory with counterexample receipts; do not promote them to accepted candidates on the basis of row count alone.

7.2 Questions with concrete next experiments

1. **Can the arbitrary-state enumerator theorem simplify the physical boot theorem?** The new interface allows arbitrary initial proof-list, candidate, and arithmetic counters once the formula/cache/stack invariant is established. Identify which binary boot conditions were proved solely to meet the old seed theorem. Removing such proof obligations is immediate in principle; removing physical initialization transitions requires a separate state-level construction.

2. **Can Context215 transport be factored into a small reusable checker?** The current proof exposes a useful schema: a finite inductive guard map, a reflected slice of incoming rows, and an exact transient relation with a positive time bound. A generic verified checker could accept these data and emit both halting directions. Any such checker must include transition-domain closure and explicit treatment of terminal behavior.

3. **Can the two remaining branch requirements in the partial fold catalog be made compatible?** The 52/154 example identifies a shared helper whose desired replacement conflicts with a live CB1 return. A profitable next search should either prove a distinguishing reachable context at that helper, route that return elsewhere with its complete contract, or alter the source so the two roles cease to share the same bit pattern. Repeating the same unguarded merge is already ruled out by the supplied valid-frame witness.

4. **Can a new source encoding reduce decoder cost without increasing the trusted proof surface?** Source-node counts and allocated-state counts diverge because engine/decoder row sharing is constrained. Evaluate proposed command reordering, code changes, or weaker calculi only after compiling their initializer, literal source, row allocation, and complete runtime contracts.

5. **How much of the current source-certificate proof can be removed by definitional equality?** The newest proposals already normalize ordinary zero-offset bisimulations and use balanced lookup. When normalized source graphs are literally equal, direct kernel-checked equality is a simpler attachment than an affine-loop contraction certificate. Affine contraction is more useful when comparing genuinely different loops; it should be introduced only for that need.

6. **Can the formal dependence on foundational axioms be made easier to audit?** Keep exact theorem-level assumption reports distinct from declarations imported into a library's global environment. The standalone native transport and ranked scheduler have closed reports. The supplied semantic bridge has now also passed a recursive check against

its exact pinned dependencies and prints closed theorem-level reports. Preserve that evidence when integrating it into the full lane.

8 Reproduction, provenance, and limits

8.1 What was executed

The following verification levels should remain separate when this contribution is integrated:

Component	Recorded check
Native transport and literal bindings	Actual Coq compilation; closed assumptions for selected endpoints
Exact repository semantic adapter	Ten-module minimal closure compiled; recursive `coqchk` of `ContextLiteralSparse` passed
Rank and opacity refactor	Ten project modules and a typed client compiled; six closed reports; recursive `coqchk` passed
32-mask Context family	Independent finite audit and ordinary all-tape block proof; no all-mask kernel theorem claimed
Two state-fold search variants	Completed scoped searches on the 22 exact inputs; raw receipts included
214-state rejected trial	Independent literal replay of four intended blocks and the 163/588-step CB1 counterexample
Complete ZFC client	Exact integration source supplied; full dependency closure not freshly compiled
Lean adaptation	Proposed in the report; no new Lean implementation or compilation supplied

The recorded toolchain is Coq 8.18.0 compiled with OCaml 4.14.1. The production lane's Rocq 9.1 build is not claimed as rerun. The scheduler's private build copies change only the standard-library import spelling from `Stdlib` to `Coq`; the adapter uses exact pinned BusyCoq and machine-source files. A tactic dependency contains declarations of global axioms, but the selected theorem-level assumption reports are closed. The presence of an imported declaration does not by itself mean that the checked endpoint uses it.

8.2 Artifact layout and replay

The archive is self-contained for the minimal proof builds and finite checks. It includes editable `report.tex` and its section inputs, the compiled PDF, `transport/`, `adapter/`, `rocq_seed_rank/`, `comparison/`, and `state_reduction/`. The last directory has the exact 22 native input tables, source sites, inherited symbolic-filter modules and notices, the nineteen-pair catalog, positive control, explicitly rejected trial, complete finite traces, scripts and evidence receipts.

From the extracted archive root, first run the fast audits:

```
python3 -B reproduce.py --out /tmp/k215-quick
```

To repeat the two minimal kernel-checked proof components, use a different fresh output directory:

```
python3 -B reproduce.py --out /tmp/k215-proofs --proofs
```

The `-full-search` and `-context-search` flags optionally repeat the two corpus searches and the inherited contextual-helper filter. The component scripts are also independently usable; see the root README. None of these minimal commands builds the full ZFC endpoint client. Rebuild the report with `pdflatex report.tex` twice. Use output directories outside the source archive so generated objects do not obscure its provenance.

8.3 Exact machine and source binding

The byte-level table audit and kernel table checks serve different purposes: the former binds delivered native files to Coq literals and known source bytes; the latter proves the finite table identities used in the theorems. Both are included. The exact native SHA-256 values are:

```
old.tm
9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2
new.tm  (Context215)
da20914ad9bda3f75de0381f2c05a7b5e67d26199b57b870a5ea3b72511ab627
```

The archive root manifest records file hashes. Per-component provenance records retain source commits, paths, licenses and exact input digests. `latest-refresh.json` records the five unchanged relevant Git blobs at the newer upstream pin; `state_reduction/evidence/upstream-refresh.json` records the scope and corpus overlap of the separately inspected latest search. No whole-diff review is implied by these targeted checks.

All final source changes are supplied additively. The remote repository was not modified or published by this task. The archive is intended for review and subsequent integration. Original source licenses and attribution records are retained; the numerical calculus, its ZFC calibration, the accepted machine, the Context215 retargetings, and existing runtime frameworks are inherited work.

This report does not claim global priority for the rank function, opacity technique, or positive-stuttering method. Its concrete contribution is their checked application to the current literal artifacts, the stronger enumerator interface, and the precisely scoped physical search. It does not prove Con(ZFC), unconditional nonhalting, or an impossibility theorem for 214-state machines.

References

[1] ProveIt contributors, *Smaller machines for Con(ZFC)*, pinned snapshot `aa66d0a33f1f8237a348899eb8095b045a502832`. Repository overview.

[2] ProveIt contributors, *Formal endpoints for the K-family candidates*; K215 verification and simplification records at the same snapshot. Formal architecture and endpoint status.

[3] ProveIt contributors, *History returns, a shallower 215-state machine, and new source calculi*, 21 October 2026. Context-return candidate and provenance.

[4] ProveIt incoming research packets, *K215 Checked Simplifications*, *K215 Certificate Refinements*, and *Affine Proof Cores and Transport*, 21 October 2026, exact files at the pinned snapshot. Incoming packet directory.

[5] Elliot Glazer and contributors, *k230-rocq*, commit `3142e4dd980f0be12f4abb636c4ae2528afc17eb`; Jeff's busy-beaver reference, commit `c132d7eae8158cd3a9850ea3eb7251158c3dd821`. Pinned formal reference; pinned machine-semantics reference.

[6] ProveIt contributors, *Automatic certificates and targeted elimination search*, and the semantic round-trip Lean update, refreshed snapshot `b0663213602b3635258150c58ffe105a2e0ba44f`. Automatic-certificate packet; semantic round-trip commit.

[7] ProveIt contributors, *EnumPerm.lean* and *SeedPerm.lean*, refreshed snapshot `b0663213602b`. Lean enumerator and compiler; Lean seed interfaces.